

A CONSOLIDATED AI DETECTION DASHBOARD FOR ACADEMIC INTEGRITY

Designing a Unified Multi-Platform Consensus Framework for Writing Review

Submitted by: MD Zahidul Islam (Honors Program Student & Research Assistant)

Research Advisor: Dr. Mohan

Institution: Honors Research Program

Project Version: 1.1.0 (Resource-Optimized Production Build)

Date: June 2026

1. Introduction & Research Background

The recent proliferation of Large Language Models (LLMs) like GPT-4, Claude, and Gemini has transformed written expression across professional and academic domains. However, in educational environments, distinguishing between student-authored work and machine-generated content has become a major challenge. Educators rely heavily on automated AI detection platforms to preserve academic integrity. However, individual AI detectors suffer from significant limitations:

- **High False-Positive Rates:** Research shows that detectors are often biased against non-native English writers, misclassifying their natural sentence structures as machine-generated.
- **Statistical Heuristics:** Detectors rely on statistical metrics such as *Perplexity* (the measure of text unpredictability) and *Burstiness* (the variation in sentence structure and length over a document). Because these heuristics vary across providers, relying on a single detector is statistically risky.
- **Platform Divergence:** An essay might receive a high risk score on one detector and a zero risk score on another, leading to confusion and unfair academic investigations.
- **Financial & Rate Constraints:** API access for professional systems is costly, and free tiers are heavily rate-limited.

The Unified Consensus Model:

To mitigate these issues, this project introduces a **Consolidated Multi-Detector Review Dashboard**. By querying multiple active detection platforms concurrently and aggregating their results into a simple average (a consensus score), we decrease the probability of false-positives and avoid relying on a single vendor's heuristic bias. If a single platform fails, the consolidated score filters it out dynamically, offering educators a fairer, multi-perspective assessment tool.

2. Technical Architecture & Workflows

The project is implemented using the **Next.js 15 (App Router)** web framework, leveraging TypeScript, Prisma ORM, and Tailwind CSS. The design focuses on a clean, modern user experience with a strict separation of concerns. The database default is PostgreSQL, with a seamless script to fall back to a local SQLite instance.

Core Functionalities & Mechanics:

- **Hybrid Text Extraction:** When a file is uploaded, the backend inspects the document type. We use the modern, class-based pdf-parse library for extracting PDF text, mammoth for parsing DOCX XML structures, and standard UTF-8 buffer decoding for plain text files.
- **Asynchronous Queue Runner (queue.ts):** Because AI detector API responses are network-bound and can take up to 15 seconds per request, calling them synchronously would result in web browser timeout errors. Instead, the server registers a database-backed job in a PENDING state, instantly redirects the user to the tracking page, and spawns an asynchronous worker process in the background. The user's page polls the status endpoint until the background runner updates the job states to completed.
- **Symmetric Encryption (encryption.ts):** To satisfy academic privacy compliance and protect API credentials, third-party keys are encrypted at rest using AES-256-GCM. The encrypted text, random Initialization Vector (IV), and authentication tag are saved in the database. Decryption happens strictly in-memory during scan execution, and raw keys are never sent to the client browser.
- **Role-Based Access Control (RBAC):** Admin pages (Provider Keys, Account Management, System Logs) are protected behind Next.js route middleware. Sessions are verified using custom JSON Web Tokens (JWT) stored in secure, HttpOnly cookies.

3. Free-Tier Resource Quota Enhancements

AI detection platforms on the free tier enforce strict usage limits. When scanning lengthy academic papers (e.g., final exams of 4,000+ words), these constraints often block scans entirely. We implemented direct adapter level optimizations to resolve these failures while preserving scan accuracy:

WasItAIGenerated Credit Optimization (402 Fix):

The WasItAIGenerated API charges exactly 1 credit per word scanned. On a free/trial tier (typically having less than 1,000 credits remaining), uploading a 4,500-word research guide will result in a 402 Insufficient Credits failure. We resolved this by introducing **word count truncation**. The adapter automatically grabs and scans only the first **800 words** (~4,800 characters) of the text. Because statistical writing heuristics (perplexity and burstiness) remain consistent throughout a document, an 800-word sample is statistically representative for AI detection, while guaranteeing the scan stays within your credit limits.

Sapling Rate Limit & Token Optimization (429 Fix):

Sapling enforces strict per-minute and per-token rate limits, resulting in frequent 429 Rate Limited errors. We updated the Sapling adapter to truncate texts to **1,500 words** to avoid hitting token limits, and integrated an **automatic retry-once mechanism**. If Sapling returns a 429 status, the background worker pauses for 3 seconds and automatically retries the request. This prevents transient rate spikes from failing the student's scan.

Detector	Heuristic Metrics	Free Limits Encountered	Our Consensus Optimization
WasItAIGenerated	Pattern probability analysis	Charges 1 credit per word (exceeds balance on long texts)	Truncates to 800 words to provide a statistically valid sample and prevent 402 errors.
Sapling	Sentence-level perplexity scores	Tokens-per-request limit & 429 per-minute rate limit	Truncates to 1500 words and triggers an auto-retry delay (3s) on 429 warnings.
Copyleaks	Sentence classification heuristics	Requires enterprise enterprise plan (returns 404 for AI)	Disabled globally (Offline Mode) to prevent false failed reviews.

4. Local Run & Configuration Instructions

To run the project on your local computer, follow these step-by-step commands in your terminal (PowerShell or Bash):

Step 1: Install Dependencies

Run the package manager in the project root to download and install node packages:

```
npm install
```

Step 2: Choose Your Database (PowerShell SQLite Fallback)

The default schema is PostgreSQL. To run locally without Docker or hosted databases, switch to a local SQLite database:

```
powershell -ExecutionPolicy Bypass -File .\switch-to-sqlite.ps1
```

Step 3: Setup Environment Configuration

Create the active environment variable file by copying the template:

```
cp .env.example .env
```

*Ensure that the **JWT_SECRET** and **ENCRYPTION_KEY** are set to secure random strings.*

Step 4: Initialize the Database

Generate the database tables and run the seeder script to create default accounts and configure providers:

```
npx prisma db push
```

```
npx prisma db seed
```

Default Seed Credentials for Testing:

The database seeder initializes the following default demo credentials:

- **Instructor/Admin Login:** admin@school.edu (Password: AdminPass123)
- **Student Reviewer 1 Login:** student1@school.edu (Password: StudentPass123)
- **Student Reviewer 2 Login:** student2@school.edu (Password: StudentPass123)

Step 5: Run the Development Server

Start the local Next.js dev server. It will run by default at port 3000:

```
npm run dev
```

Open <http://localhost:3000> in your browser.

5. Production Deployment & Hosting Guide

To share the application with other students or faculty, you can deploy it online. Since Next.js requires a Node.js server environment for the queue worker and text extractors, we have two primary hosting options:

Option A: Vercel (Frontend/Serverless) + Supabase/Neon (PostgreSQL)

This is the easiest and most cost-effective hosting option. Vercel hosts the Next.js app in a serverless environment.

1. Create a free PostgreSQL database on **Supabase** (supabase.com) or **Neon** (neon.tech).
2. Get your connection string (e.g., postgres://user:pass@host:5432/db).
3. Push the project to GitHub (excluding the .env file).
4. Link your repository to Vercel (vercel.com). Under settings, set these environment variables:
 - DATABASE_URL: Set to the transaction connection string of your PostgreSQL database.
 - DIRECT_URL: Set to the session connection string of your PostgreSQL database (for migrations).
 - JWT_SECRET: A random 32-character string for securing student sessions.
 - ENCRYPTION_KEY: A random 32-byte hex string (64 characters) used for encrypting API keys at rest.
5. Set Vercel's build command to run migrations before building: `npx prisma db push && next build`.

Option B: Self-Hosting on Render or Railway (Docker/VPS)

If you want to continue using the simple SQLite file without setting up a hosted PostgreSQL database:

1. Deploy the app to **Render.com** or **Railway.app** as a Web Service.
2. Add a **Persistent Volume/Disk** and mount it to the directory containing your database (e.g., /app/prisma/dev.db). This ensures your student accounts, settings, and logs aren't deleted when the server restarts.
3. Configure the environment variables in their dashboard settings.
4. Use build command: `npm install && npx prisma db push && npx prisma db seed && npm run build` and start command: `npm run start`.